

CS456 Design and Analysis of Algorithms

Project 1 - Interval Scheduling

Deepshan Adhikari (800846035)
deepadh@siue.edu

1. Introduction

The Interval Scheduling Problem is a fundamental optimization problem in computer science. Given a set of jobs where each job has a start time and a finish time, the objective is to select the largest possible subset of jobs such that no two jobs overlap. Two jobs are compatible if one finishes before the other starts. This problem appears in many real world applications including CPU task scheduling, resource allocation, and network bandwidth management.

This project compares four greedy strategies for solving the Interval Scheduling Problem as described in Kleinberg and Tardos [1], Algorithm Design, pages 116 to 118. The four strategies are:

- a. **Earliest Finish Time First (EFT):** This algorithm sorts jobs by finish time from earliest to latest. It scans the sorted list and selects a job if its start time is at or after the finish time of the previously selected job. This strategy is proven optimal by Theorem 4.1 in Kleinberg and Tardos [1] because the earliest finishing job always leaves maximum room for future jobs.
- b. **Shortest Duration First (SDF):** This algorithm sorts all jobs by their duration from shortest to longest. It then scans the sorted list and selects a job only if it does not overlap with any previously selected job. This heuristic has no optimality guarantee because a short job can block two longer non-overlapping jobs on either side.
- c. **Earliest Start Time First (EST):** This algorithm sorts all jobs by their start time from earliest to latest. It then scans the sorted list and selects a job if its start time is at or after the finish time of the previously selected job. This heuristic also has no optimality guarantee because an early-starting job may finish late and block many other jobs.
- d. **Fewest Conflicts First (FCF):** This algorithm first counts how many other jobs each job overlaps with, then sorts jobs by this conflict count from lowest to highest. It scans the sorted list and selects a job only if it does not overlap with any previously selected job. This heuristic is described on page 118 of Kleinberg and Tardos [1] and is not provably optimal because low-conflict jobs are not guaranteed to be mutually compatible with each other.

The experiment was implemented in C++. The experiment runs 1000 trials, each with 1000 randomly generated jobs. All strategies run on the same job array per trial for accurate comparison. Results are reported as high, mean, and low values across all trials.

2. Results

Figure 1 summarizes the results of running all four strategies across 1000 trials, where each trial worked with its own set of 1000 randomly generated jobs. All four strategies were applied to the same job array in each trial.

```
CS456 Design and Analysis of Algorithms
Project 1 - Interval Scheduling

Running 1000 trials (1000 jobs per trial)...

Results after 1000 trials (1000 jobs per trial):
```

	High Value	Mean Value	Low Value
Earliest Finish Time (EFT)	115	100.86	88
Shortest Duration First (SDF)	113	98.78	86
Earliest Start Time (EST)	49	36.58	28
Fewest Conflicts First (FCF)	115	99.47	86

```
Run with greatest performance gap between strategies: Run 113

EFT = 115
SDF = 113
EST = 36
FCF = 114

Performance Gap (max - min) = 115 - 36 = 79
```

Figure 1: Experimental Results for Four Interval Scheduling Strategies.

3. Discussion

3.1 Expected Results

Earliest Finish Time (EFT) is the only strategy among the four with a formal correctness guarantee. Theorem 4.1 in Kleinberg and Tardos [1] proves using a "stays ahead" argument that the greedy algorithm which always selects the job with the earliest finish time produces an optimal schedule. Therefore, EFT should consistently produce the highest number of scheduled jobs across all trials. Since EFT is optimal for every input, it should have the highest high, mean, and low values.

Shortest Duration First (SDF) is known to be suboptimal. On random inputs where jobs are spread across a wide time range, short jobs are often mutually compatible. However, a short job positioned between two non-overlapping longer jobs can block both, resulting in fewer total scheduled jobs. SDF is expected to perform reasonably well but not as well as EFT.

Earliest Start Time First (EST) is also not optimal. An early-starting job may have a very late finish time, blocking many shorter compatible jobs that could otherwise have been scheduled. EST is expected to perform noticeably worse than EFT and likely worse than SDF.

Fewest Conflicts First (FCF) is described on page 118 of Kleinberg and Tardos [1] as a heuristic that sorts jobs by the number of conflicts. The intuition is that low-conflict jobs are less likely to block future selections. FCF is expected to perform well, potentially close to EFT on average, though it lacks a formal optimality guarantee.

3.2 Comparison of Results to Expectations

The experimental results line up with what was expected. Earliest Finish Time (EFT) performed best with a mean of 100.86 and a high of 115, confirming its optimality guarantee from Kleinberg and Tardos [1].

Earliest Start Time (EST) was the weakest by a large margin, averaging only 36.58 jobs. The other three strategies all averaged between 98 and 101 jobs, meaning EST fell roughly 60 to 65 jobs behind on average. This confirms that picking jobs solely by start time is a poor strategy, as an early starting job can have a late finish time and block many others.

Shortest Duration (SDF) averaged 98.78, while Fewest Conflicts (FCF) averaged 99.47. Both came within about 1 to 2 jobs of EFT, showing that these heuristics work well on random inputs even without an optimality guarantee.

3.3 Ranking of the Methods

Earliest Finish Time is the best strategy with a mean of 100.86. It consistently outperformed the other three strategies across all 1000 trials. This was expected since it is the only strategy with a formal proof of optimality from Theorem 4.1 in Kleinberg and Tardos [1].

Fewest Conflicts is second with a mean of 99.47. It performed very close to EFT, coming within about 1.4 jobs on average. This suggests that sorting jobs by the number of conflicts is a strong heuristic in practice, even though it lacks a formal optimality guarantee.

Shortest Duration is third with a mean of 98.78. It performed slightly behind FCF but still came within about 2 jobs of EFT on average. This makes sense because shorter jobs naturally take up less time, making them more likely to fit together without overlapping.

Earliest Start Time is last with a mean of only 36.58. It scheduled roughly a third of what the top strategies achieved on average. This large gap confirms that sorting by start time is not a useful criterion for interval scheduling, as an early starting job with a late finish time can block many other jobs.

3.4 A Counterexample: Where Earliest Start Time is Not Optimal

Consider the following three jobs:

Jobs	Start	Finish	Duration
A	1	100	99
B	2	5	3
C	6	9	3

This example, taken from Kleinberg and Tardos [1], demonstrates why Earliest Start Time is not optimal. Earliest Start Time picks Job A first because it starts at time 1, which is the earliest. Since Job A runs from time 1 to time 100, it overlaps with both Job B and Job C. As a result, neither B nor C can be scheduled, and EST produces only 1 job.

The optimal schedule is to pick Job B and Job C. Job B finishes at time 5, and Job C starts at time 6, so they are compatible. This gives 2 jobs, which is optimal.

Earliest Finish Time finds this optimal schedule. It picks Job B first since it finishes earliest at time 5, then picks Job C since it starts after B finishes, and skips Job A because it overlaps with both. EFT schedules 2 jobs.

This shows that EST fails because an early starting job can have a very late finish time, blocking other jobs that could have been scheduled. EFT succeeds because it prioritizes jobs that finish early, leaving more room for others.

3.5 One Experimental Run Where the Strategies Differed Significantly

Run 113 is a good example of just how far apart these strategies can land on the same input. On that trial, Earliest Start Time scheduled only 36 jobs, while Earliest Finish Time reached 115, Fewest Conflicts reached 114, and Shortest Duration reached 113. This created a performance gap of 79 jobs between the best and worst strategies, all working from the exact same set of 1000 randomly generated jobs.

The reason comes down to the same problem shown in the small example above, just playing out at a much larger scale. With 1000 jobs spread across a wide range of start times, it is common for a few of them to happen to start early while also running for a long stretch. Earliest Start Time grabs those jobs right away since it is only looking at when they begin, and each one it locks in can end up blocking a large number of shorter jobs that would have fit together nicely later on. The other three strategies avoid this trap because they are each, in their own way, paying attention to how much of the timeline a job actually takes up or how many other jobs it collides with, not just where it happens to start.

3.6 Why Correctness Proofs Matter for Greedy Algorithms

This experiment shows exactly why a greedy algorithm needs an actual proof behind it, not just strong performance on test data. All four strategies here sound reasonable on paper, and on this particular random data, three of the four ended up performing close to each other. But only Earliest Finish Time comes with a guarantee that holds for every possible input. The other three can still fail badly on certain inputs, as shown in sections 3.4 and 3.5, even though they looked solid and performed well most of the time.

Without proof, there is no way to know ahead of time which of these strategies is actually safe to depend on. A strategy that averages well but occasionally collapses, the way Earliest Start Time did in Run 113, could cause real problems if used somewhere that needed consistent results. A correctness proof rules that out completely, since it holds for every input, not just the ones that happened to get tested. That is something no amount of experimental data alone can ever fully guarantee.

4. Code

The following C++ program implements all four interval scheduling strategies and runs the experiment described in this report.

```
// CS456 Design and Analysis of Algorithms
// Project 1 - Interval Scheduling

// Author: Deepshan Adhikari (800846035)
// deepadh@siue.edu
// September 6, 2026

// Description:
// This program compares four different interval scheduling strategies,
// by running 1000 trials with 1000 random jobs each.

// It analyzes:
// 1. Earliest Finish Time (EFT) - PROVEN OPTIMAL
// 2. Shortest Duration First (SDF) - NOT OPTIMAL
// 3. Earliest Start Time (EST) - NOT OPTIMAL
// 4. Fewest Conflicts First (FCF) - NOT OPTIMAL
//
// The program generates random intervals (jobs), applies each strategy,
// and reports the maximum, mean, and minimum number of jobs that can
// be scheduled for each strategy.

#include <iostream> // For input/output operations
#include <vector>    // For using dynamic arrays (vectors)
#include <algorithm> // For sorting algorithms
```

```

#include <random>    // For generating random numbers
#include <iomanip>    // For formatting output (setw, setprecision)
#include <climits>   // For using INT_MAX/INT_MIN constants

using namespace std;

// Job Structure
// Represents an interval/job with start time, duration, and finish time
struct Job
{
    int start;    // Start time of the job
    int duration; // Duration/length of the job
    int finish;   // Finish time (start + duration)

    // Constructor: automatically calculates finish time
    Job(int s = 0, int d = 0) : start(s), duration(d), finish(s + d) {}
};

// Creates an array of random jobs
// Parameters: n - Number of jobs to create (default: 1000)
// Returns: Vector containing n random jobs
vector<Job> createJobArray(int n = 1000)
{
    random_device rd; // Creates a random number
generator using real randomness from the laptop device to get a truly random seed
    mt19937 gen(rd()); // Creates a random number
engine that generates millions of random numbers quickly
    uniform_int_distribution<> startDist(1, 9500); // Creates a rule "give any
whole number between 1 and 9500, all equally likely" - used for job start times
    uniform_int_distribution<> durationDist(10, 500); // Creates a rule "give any
whole number between 10 and 500, all equally likely" - used for how long each job
takes

    vector<Job> jobs;
    jobs.reserve(n); // Reserve memory for efficiency

    // Create n random jobs
    for (int i = 0; i < n; i++)
    {
        int start = startDist(gen);
        int duration = durationDist(gen);
        jobs.push_back(Job(start, duration));
    }
    return jobs;
}

```

```

// Chronological Scheduler
// Used for EFT and EST strategies where jobs are sorted by time
// Only checks against the last accepted job (greedy approach)
// Parameters: sortedJobs - Jobs sorted by finish time or start time
// Returns: Number of jobs that can be scheduled
int scheduleChronological(vector<Job> sortedJobs)
{
    int lastFinish = 0; // Finish time of the last accepted job
    int count = 0;      // Number of accepted jobs

    // Check each job in order
    for (const auto &job : sortedJobs)
    {
        // If job starts after or when last job finished, accept it
        if (job.start >= lastFinish)
        {
            count++;
            lastFinish = job.finish; // Update last finish time
        }
    }
    return count;
}

// General Scheduler
// Used for SDF and FCF strategies
// Checks compatibility with ALL previously accepted jobs
// Parameters: sortedJobs - Jobs sorted by duration or conflict count
// Returns: Number of jobs that can be scheduled
int scheduleGeneral(vector<Job> sortedJobs)
{
    vector<Job> accepted; // List of accepted jobs

    // Check each job in order
    for (const auto &job : sortedJobs)
    {
        bool compatible = true;

        // Check if job overlaps with any accepted job
        for (const auto &a : accepted)
        {
            // Check for overlap: Two jobs overlap if one starts before the other
            // finishes and vice versa
            // job1.start < job2.finish && job2.start < job1.finish
            if (job.start < a.finish && a.start < job.finish)

```

```

        {
            compatible = false;
            break; // No need to check further
        }
    }

    // If compatible, accept the job
    if (compatible)
        accepted.push_back(job);
}
return accepted.size();
}

// Strategy 1: Earliest Finish Time First
// Sorts jobs by finish time and schedules greedily
// PROVEN OPTIMAL - This always gives the maximum number of jobs
int earliestFinishTimeFirst(vector<Job> jobs)
{
    // Sort by finish time (ascending)
    sort(jobs.begin(), jobs.end(), [](const Job &a, const Job &b)
        { return a.finish < b.finish; });
    return scheduleChronological(jobs);
}

// Strategy 2: Shortest Duration First
// Sorts jobs by duration and schedules non-overlapping ones
// NOT OPTIMAL - May fail to find maximum number of jobs
int shortestDurationFirst(vector<Job> jobs)
{
    // Sort by duration (shortest first)
    sort(jobs.begin(), jobs.end(), [](const Job &a, const Job &b)
        { return a.duration < b.duration; });
    return scheduleGeneral(jobs);
}

// Strategy 3: Earliest Start Time First
// Sorts jobs by start time and schedules greedily
// NOT OPTIMAL - May fail to find maximum number of jobs
int earliestStartTimeFirst(vector<Job> jobs)
{
    // Sort by start time (earliest first)
    sort(jobs.begin(), jobs.end(), [](const Job &a, const Job &b)
        { return a.start < b.start; });
    return scheduleChronological(jobs);
}

```

```

// Count Conflicts for FCF Strategy
// For each job, count how many other jobs overlap with it
// Parameters: jobs - Array of jobs
// Returns: Vector where conflicts[i] = number of jobs overlapping with job i
vector<int> countConflicts(const vector<Job> &jobs)
{
    int n = jobs.size();
    vector<int> conflicts(n, 0); // Initialize all conflicts to 0

    // Compare each pair of jobs
    for (int i = 0; i < n; i++)
    {
        for (int j = i + 1; j < n; j++)
        {
            // Check if job i and job j overlap
            if (jobs[i].start < jobs[j].finish && jobs[j].start < jobs[i].finish)
            {
                conflicts[i]++; // Job i has one more conflict
                conflicts[j]++; // Job j has one more conflict
            }
        }
    }
    return conflicts;
}

// Strategy 4: Fewest Conflicts First
// Sorts jobs by number of conflicts (fewest first)
// NOT OPTIMAL - Graduate requirement, may fail to find maximum
int fewestConflictsFirst(vector<Job> jobs)
{
    // Count conflicts for each job
    vector<int> conflicts = countConflicts(jobs);

    // Create pairs of (conflict_count, job)
    vector<pair<int, Job>> jobsWithConflicts;
    for (int i = 0; i < jobs.size(); i++)
    {
        jobsWithConflicts.push_back({conflicts[i], jobs[i]});
    }

    // Sort by conflict count (fewest conflicts first)
    sort(jobsWithConflicts.begin(), jobsWithConflicts.end(),
        [](const pair<int, Job> &a, const pair<int, Job> &b)
        {

```

```

        return a.first < b.first; // Ascending order of conflicts
    });

    // Extract just the jobs (without conflict counts)
    vector<Job> sortedJobs;
    for (const auto &p : jobsWithConflicts)
    {
        sortedJobs.push_back(p.second);
    }

    return scheduleGeneral(sortedJobs);
}

// Main Function
// Runs the experiment, collects data, and displays results
int main()
{
    // Experiment setup
    const int RUNS = 1000; // Number of trials
    const int JOBS = 1000; // Jobs per trial

    // Vectors to store results for each strategy
    vector<int> eftResults, sdfResults, estResults, fcfResults;
    eftResults.reserve(RUNS);
    sdfResults.reserve(RUNS);
    estResults.reserve(RUNS);
    fcfResults.reserve(RUNS);

    // Variables to track the most notable run (largest performance gap)
    int maxDifference = 0;
    int notableRun = 0;
    int notableEFT = 0, notableSDF = 0, notableEST = 0, notableFCF = 0;

    // Display header
    cout << "\nCS456 Design and Analysis of Algorithms" << endl;
    cout << "Project 1 - Interval Scheduling" << endl;
    cout << "\nRunning 1000 trials (1000 jobs per trial)..." << endl;

    // Run experiment
    for (int run = 0; run < RUNS; run++)
    {
        // Generate random jobs for this trial
        vector<Job> jobs = createJobArray(JOBS);

        // Apply all four strategies

```

```

    int eft = earliestFinishTimeFirst(jobs);
    int sdf = shortestDurationFirst(jobs);
    int est = earliestStartTimeFirst(jobs);
    int fcf = fewestConflictsFirst(jobs);

    // Store results
    eftResults.push_back(eft);
    sdfResults.push_back(sdf);
    estResults.push_back(est);
    fcfResults.push_back(fcf);

    // Check if this run has the largest performance gap
    int difference = max(max(eft, sdf), max(est, fcf)) -
                     min(min(eft, sdf), min(est, fcf));
    if (difference > maxDifference)
    {
        maxDifference = difference;
        notableRun = run + 1;
        notableEFT = eft;
        notableSDF = sdf;
        notableEST = est;
        notableFCF = fcf;
    }
}

// Function to calculate high, mean and low value for a dataset
auto summarize = [](const vector<int> &v)
{
    int high = *max_element(v.begin(), v.end());
    double mean = 0;
    for (int x : v)
        mean += x;
    mean /= v.size();
    int low = *min_element(v.begin(), v.end());
    return make_tuple(high, mean, low);
};

// Get statistics for each strategy
auto [eftHigh, eftMean, eftLow] = summarize(eftResults);
auto [sdfHigh, sdfMean, sdfLow] = summarize(sdfResults);
auto [estHigh, estMean, estLow] = summarize(estResults);
auto [fcfHigh, fcfMean, fcfLow] = summarize(fcfResults);

cout << "\nResults after 1000 trials (1000 jobs per trial):" << endl;
cout << "\n"

```

```

        << setw(30) << "" << setw(14) << "High Value"
        << setw(14) << "Mean Value" << setw(12) << "Low Value" << endl;
    cout << string(75, '-') << endl;

    cout << left << setw(32) << "Earliest Finish Time (EFT)"
        << right << setw(10) << eftHigh
        << setw(16) << fixed << setprecision(2) << eftMean
        << setw(10) << eftLow << endl;

    cout << left << setw(32) << "Shortest Duration First (SDF)"
        << right << setw(10) << sdfHigh
        << setw(16) << fixed << setprecision(2) << sdfMean
        << setw(10) << sdfLow << endl;

    cout << left << setw(32) << "Earliest Start Time (EST)"
        << right << setw(10) << estHigh
        << setw(16) << fixed << setprecision(2) << estMean
        << setw(10) << estLow << endl;

    cout << left << setw(32) << "Fewest Conflicts First (FCF)"
        << right << setw(10) << fcfHigh
        << setw(16) << fixed << setprecision(2) << fcfMean
        << setw(10) << fcfLow << endl;

    cout << "\nRun with greatest performance gap between strategies: Run " <<
notableRun << endl;
    cout << "\nEFT = " << notableEFT << endl;
    cout << "SDF = " << notableSDF << endl;
    cout << "EST = " << notableEST << endl;
    cout << "FCF = " << notableFCF << endl;
    cout << "\nPerformance Gap (max - min) = " << max({notableEFT, notableSDF,
notableEST, notableFCF})
        << " - " << min({notableEFT, notableSDF, notableEST, notableFCF})
        << " = " << maxDifference << endl;

    return 0;
}

```

5. Conclusion

This experiment demonstrates that not all greedy algorithms are optimal for the Interval Scheduling Problem. Earliest Finish Time, which has a formal proof of optimality from Kleinberg and Tardos [1], consistently outperformed the other three strategies, averaging 100.86 jobs scheduled across 1000 trials. Earliest Start Time performed the worst by a large margin, averaging only 36.58 jobs, confirming that sorting by start time is a poor strategy for interval scheduling.

The results also show that Shortest Duration and Fewest Conflicts, while not provably optimal, perform nearly as well as Earliest Finish Time on random inputs. Fewest Conflicts averaged 99.47 and Shortest Duration averaged 98.78, both coming within about 1 to 2 jobs of EFT. However, this does not guarantee they will work on all inputs, as shown by the counterexample in section 3.4.

This experiment highlights why correctness proofs are essential in algorithm design. Without a proof, there is no way to guarantee an algorithm will work correctly for all possible inputs. That guarantee is something experimental testing alone can never fully provide.

References

[1] Kleinberg, Jon & Tardos, Éva. (2006). Algorithm Design.